Security Posture Scorecard

Solution Brief — github.com/him-agni/security-posture-scorecard

Problem: Static security scanners generate too much noise, and teams stop trusting the results. Worse, some scanners report false-green passes on unverifiable controls like encryption-at-rest — hiding real risk under a clean-looking scorecard. Developers dismiss findings, and important repos go unpatched because the report looked fine.

Solution: A confidence-tiered scorecard that labels every finding as 'verified,' 'detected,' or 'manual' — so teams know exactly what was proven from source, what was heuristically inferred, and what still needs a human to confirm. The result is a scorecard developers actually trust and act on.

How It Works: Repo URL -> Octokit fetches tarball to temp dir -> file-tree context built -> registered checks run across frontend, backend, and database layers -> severity-weighted scorer -> per-layer + overall grade -> temp dir cleaned up.

Key Capabilities

- Three-tier confidence labels (verified / detected / manual) on every finding
- Plugin architecture — each check is a self-contained module, so new checks are additive
- Transparent scoring with a 'why this score' breakdown, not a black-box grade
- OSV database lookup for known-vulnerable dependencies from the lockfile
- Not-applicable detection, so a frontend library isn't penalized for missing backend controls
- Manual checklist for unverifiable-from-source items (encryption at rest, backups), never faked as green

Business Outcome: Security scores developers trust and act on, no false greens hiding real gaps, and faster remediation prioritization because the confidence tier tells teams which findings need immediate action vs. human review.

Tech Stack: React, Vite, Node.js, Express, Octokit (GitHub API), OSV vulnerability database, node:test, Vitest, Playwright.